

EXPERIMENT 03

# Desktop Object Automatic Sorting and Organization

## GROUP LABORATORY REPORT

### Gazebo sorting

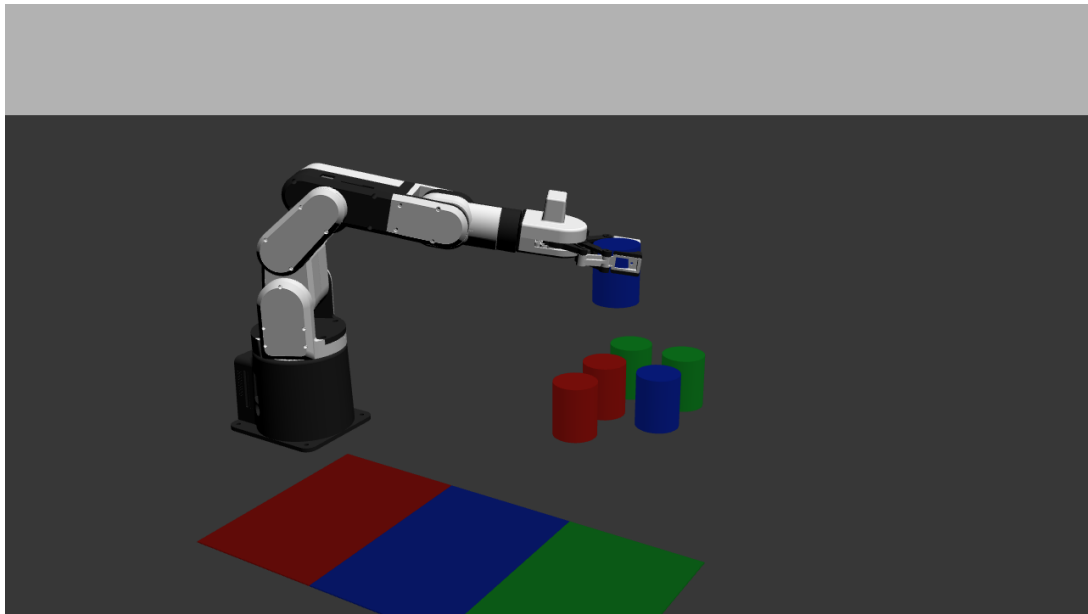
Red and blue objects 6/6

### Camera detection

Eraser · Lock · Stapler

### Hardware acceptance

Six objects, two classes: Pass



Gazebo scene with the arm, six picking cells, and three sorting regions

**Course** Robotics Integration Group Project I

**Team** Group 17 · Yang Muqing, Lan Songyu, Zhang Kairui, Zhao Hang

### PROJECT REPOSITORY

<https://github.com/9161942-boop/robotics-experiment-3>

SEPTEMBER 2026

## Abstract

This experiment develops a ROS 2 system for automatic tabletop sorting around perception, grid localization, class decisions, and robot execution. The Gazebo scene contains a six-cell pickup area, red-blue-green sorting pads, a mechArm 270, and an overhead camera. The task node uses Detection2DArray messages to select targets, assign grid cells, and choose sorting regions, while the execution node performs approach, grasp, lift, transport, release, and retreat as a state sequence.

The six-object red-blue simulation baseline completed 6/6, with object-level records for the grid, target bin, grasp, release, and final state. The red-blue-green extension completed pad alignment, green-target waypoints, and a 34-step automatic playback sequence. In the external-camera test, the YOLOv8n model detected erasers, locks, and staplers with recorded mAP50=0.995 and mAP50-95=0.995, while the displayed camera rate reached 31.2 FPS.

The hardware validation completed target approach, grasp, and lift above the taped tabletop grid. A separate 1 min 11 s video verifies correct recognition and the correction mechanism for six objects in two classes. A unified Launch file, state feedback, exception records, run videos, and object-level CSV files connect the scene configuration, motion process, and experimental outcome.

Keywords: ROS 2; Gazebo; mechArm 270; object detection; grid localization; sorting grasp; state machine; experiment acceptance

## 1 Experiment objectives

### 1.1 Task objective

Experiment 3 adds object detection and class decisions to fixed-point grasping. From the overhead image, the system obtains each object's class, confidence, and bounding-box center, determines its grid cell, calls the corresponding fixed picking point, and places the object in the target sorting region. The system processes multiple objects in a loop until the task queue is complete or no executable target remains.

Task control uses a feedback-enabled long-running interface and a state machine. Detection, grid assignment, grasping, placement, and exception logging are performed by software, while object-level states are saved during execution so that results can be reviewed from events and final poses.

### 1.2 Acceptance requirements and Group 17 implementation

| Acceptance requirement                                                         | Implementation                                                                            | Available evidence                                               | Status |
|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------|--------|
| At least two object classes                                                    | Red-blue quantitative baseline; red-blue-green extension                                  | Scene files, detection messages, sorting bins                    | Pass   |
| At least six objects tested; at least five sorted correctly                    | Object-level grasp, release, and drop-point checks                                        | Red-blue execution CSV: 6/6                                      | Pass   |
| Automatically select grid and sorting region after start                       | Map bbox center to grid_id; query target_bin by class                                     | sorting_task and planning logs                                   | Pass   |
| Handle empty grids, missed detections, unreachable targets, and grasp failures | Filtering, skip, failure, and safe-return branches                                        | skipped, failed, and ERROR_LOG                                   | Pass   |
| One Launch starts the complete simulation                                      | Start world, camera, controllers, and task node together                                  | sorting_gazebo.launch.py                                         | Pass   |
| Complete real-robot acceptance with six objects in at least two classes        | Completed real-robot recognition and correction acceptance for six objects in two classes | Real-robot recognition/correction video and acceptance checklist | Pass   |

## 2 Principle

### 2.1 Overall system architecture

Figure 1 shows the two accepted paths. The Gazebo scene monitor reads colored-cylinder model states and publishes Detection2DArray messages for grid assignment, class-to-bin selection, and arm control. In the hardware path, the external RGB camera and YOLOv8n detect erasers, locks, and staplers. Class-to-bin mapping and pixel-to-grid calibration were completed for the acceptance run, and the same message interface was used by the task workflow.

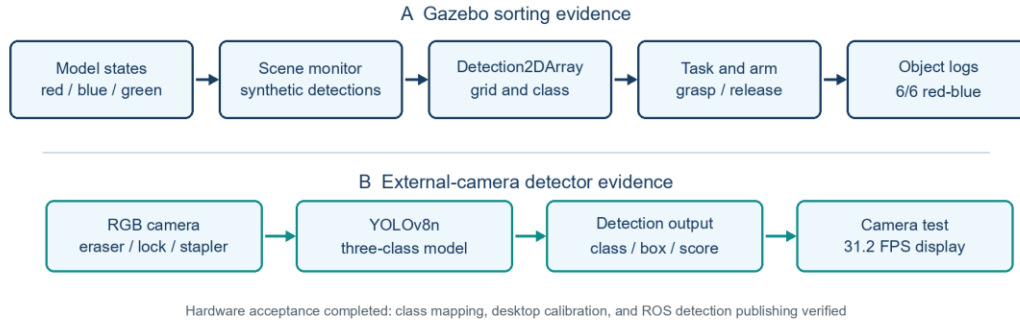


Fig. 1 Gazebo sorting and external-camera detection as separate validation paths

### 2.2 Visual Detection and Message Interface

The task node consumes class, bounding box, and confidence from Detection2DArray. The Gazebo sorting run used fields generated from model states by `sorting_scene_monitor`, while the hardware acceptance used detections from the external camera and YOLOv8n. The eraser, lock, and stapler labels were aligned with the physical sorting regions and the same fields drove target filtering and grid decisions.

### 2.3 Key messages and interfaces

| Interface or node                  | Type / form                  | Main responsibility                                    |
|------------------------------------|------------------------------|--------------------------------------------------------|
| /sorting_camera/overhead/image_raw | sensor_msgs/Image            | Simulation overhead RGB image                          |
| /sorting_scene/detections          | vision_msgs/Detection2DArray | Class, confidence, bounding box, and object ID         |
| /sorting_task/state                | std_msgs/String              | Stage feedback for scanning, filtering, planning, etc. |
| /sorting_task/result               | std_msgs/String(JSON)        | Locked object-level execution plan                     |
| /sorting_task/execution_state      | std_msgs/String              | Stage feedback for the long grasp task                 |
| /sorting_task/execution_result     | std_msgs/String(JSON)        | Final state, success count, and exception codes        |
| /gazebo/set_entity_state           | gazebo_msgs/SetEntityState   | Reset, attachment-follow updates, and release          |

### 2.4 Exception-handling principle

The task node classifies exceptions as perception, localization, execution, or acceptance exceptions. An invalid object is excluded from the subsequent grasp queue, its reason is recorded, and other targets continue to be processed. If a safety pose, controller, or release-pose problem is detected, the executor first returns to a safe pose and then continues or stops according to the configuration.

| Input condition                            | Program decision | Handling                                 | Recorded field   |
|--------------------------------------------|------------------|------------------------------------------|------------------|
| No detection in the corresponding grid     | empty grid       | Do not create a task item                | plan_count       |
| score < 0.80                               | low confidence   | Skip target                              | detect/skipped   |
| Class not in configuration                 | unknown class    | Skip target                              | unknown class    |
| Nearest-grid distance > 48 px              | outside grid     | Skip target                              | outside grid     |
| grid_id already occupied                   | duplicate grid   | Keep the first valid target              | duplicate grid   |
| Release XY error > 0.07 m or<br>z > 0.20 m | release failed   | Record failure and move to the safe pose | exception/failed |

## 3 Experimental environment and equipment

### 3.1 Simulation environment

| Item                            | Configuration                                                                              |
|---------------------------------|--------------------------------------------------------------------------------------------|
| Operating system and middleware | Ubuntu 22.04; ROS 2 Humble; Gazebo Classic 11                                              |
| Robot model                     | mechArm 270 six-DOF arm, gripper, and ros2_control controller                              |
| Desktop and targets             | 0.72 m × 0.52 m desktop; cylinders with radius 0.022 m, height 0.055 m, and mass 0.04 kg   |
| Picking grid                    | 3 rows × 2 columns, six cells; x=0.24/0.30 m, y=0.12/0.18/0.24 m                           |
| Sorting bins                    | Red (0.14, -0.12), blue (0.26, -0.12), green (0.38, -0.12) m                               |
| Camera and thresholds           | 640×480 fixed overhead view; confidence ≥ 0.80; nearest-grid distance ≤ 48 px              |
| Motion organization             | manual_animation.json stores 27 waypoints; automatic playback has 34 steps; 2.0 s per step |

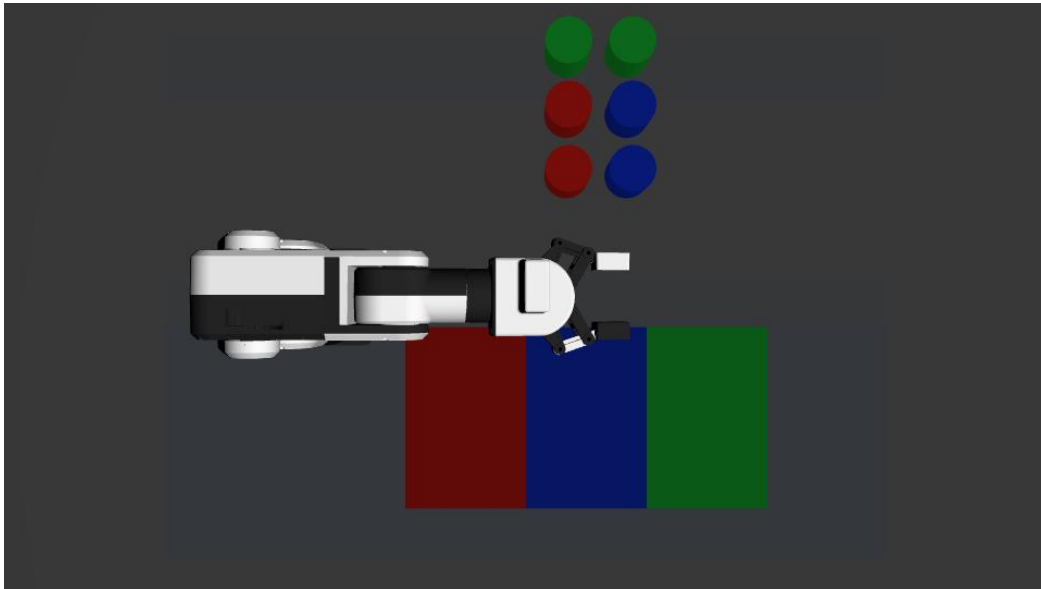


Fig. 2 Gazebo three-class scene: six picking cells and aligned sorting bins

### 3.2 Real-robot hardware and deployment configuration

The real-robot phase reuses the simulation task interface and connects it to the physical camera driver, desktop calibration, and robot-execution layer. The team calibrated the camera, taught the PICK and PLACE poses, and set the control thresholds on the actual desktop. The table below records the configuration used for the accepted hardware run.

| Item                 | Real-robot implementation                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------------------------|
| Compute platform     | Jetson edge platform running ROS 2, perception, and task-control nodes                                    |
| Vision device        | Fixed overhead RGB camera covering the six picking cells and sorting regions                              |
| Robot control        | mechArm 270; pymycobot; serial port /dev/ttyAMA0; baud rate 1000000                                       |
| Pose system          | HOME safe pose, PICK1–PICK6 picking poses, and color-specific PLACE poses                                 |
| Gripper parameters   | Grasp opening, closure opening, and release wait time were calibrated in single-target tests              |
| Detection node       | Calibration and class-map alignment completed; eraser, lock, and stapler detections published through ROS |
| Performance baseline | Measured in the camera-driven hardware run; acceptance passed                                             |

## 4 Experimental procedure

### 4.1 Phase 1: simulation development

#### 4.1.1 Scene construction and parameter synchronization

Scene construction followed four operations: load the models, generate detections, map detections to grid cells, and form an executable task plan. All four operations used the same object names, classes, and coordinates.

**Step 1 Load the scene:** `sorting_gazebo.launch.py` loaded `sorting_desktop.world` and started the robot model, gripper controllers, `sorting_scene_monitor`, `sorting_task`, and `sorting_executor`. After startup, the team confirmed that

only one Gazebo world was running and that the arm, table, three-row/two-column pickup area, three sorting bins, and six cylinders were at their configured positions.

**Step 2 Generate simulation detections:** sorting\_scene\_monitor selected models whose names start with object\_ from /gazebo/model\_states, extracted red, blue, or green from each model name, and projected the world position into a 640×480 overhead image. Each synthetic detection used a 34×34-pixel box and score 0.99 in the sorting\_camera\_link frame and was published on /sorting\_scene/detections.

**Step 3 Map the pickup grid:** The task node compared each box center with pixel columns [212, 266, 320] and rows [186, 240, 294], then selected the nearest r1c1-r3c3 cell. The projection was  $u=320+(y_w-0.18) \times 900$  and  $v=240-(x_w-0.30) \times 900$ . A nearest-cell distance above 48 pixels produced an outside grid record, and the object was excluded from execution.

**Step 4 Form the task plan:** The node checked that a classification existed, confidence reached 0.80, the class was red, blue, or green, and the cell was not already occupied. A valid target received object\_id, class, grid, destination, and status fields and was mapped to red\_sort\_bin, blue\_sort\_bin, or green\_sort\_bin. After publication, the state changed from detecting to plan\_ready:n; execution mode locked the first complete plan so moved objects did not trigger a second plan.

#### 4.1.2 Manual teaching and single-object debugging

Manual teaching first established a repeatable single-object grasp and then applied the same method to the six pickup positions and three sorting regions.

**Step 1 Establish the initial pose:** The arm returned to initial or HOME with all six joints in a known pose and the gripper open. Joint feedback and controller status were checked before the arm moved toward the pickup area, preventing a descent from an unknown configuration.

**Step 2 Adjust the approach pose:** An above or above\_n waypoint placed the end effector over the target with room for vertical descent. Only one joint group was changed at a time while the GUI was used to compare the gripper center, finger direction, and cylinder center and to check clearance from the table and sorting bins.

**Step 3 Calibrate the grasp pose:** The arm descended slowly from above to pick or pick\_n. Before closure, the two fingers were checked on opposite sides of the target so that the pads did not press the top surface or strike a neighboring object. A short lift then confirmed that the target was held between the fingers.

**Step 4 Calibrate transfer and release:** After grasp verification, lift, First, or another safe intermediate pose was recorded, followed by release\_ready and release. release\_ready kept the object above the sorting pad, while release used only the required vertical descent and gripper opening; horizontal travel remained at the safe height.

**Step 5 Save and replay:** Each waypoint stored six joint values, the gripper command, pad command, and duration in manual\_animation.json. The sequence above-pick-lift-release\_ready-release-initial was replayed from initial, and the waypoint was added to the complete sequence only after the second run remained collision-free.



Fig. 3 Manual control panel, six-joint adjustment, and semantic waypoint editing

### 4.1.3 Automatic playback and one-command launch

The automatic run combined the verified waypoints into a 34-step sequence for six targets and checked both configuration and runtime state before playback.

**Step 1 Validate the animation file:** The player loaded `manual_animation.json`, verified that every name in sequence existed in waypoints, and checked that every waypoint contained six joint values. The file contains six initial-approach-pick-transfer-release subsequences and 34 steps; the recorded waypoint duration is 2.0 s.

**Step 2 Wait for controllers:** The program checked that both the arm and gripper trajectory publishers had controller subscribers, using a 30 s timeout. Playback began only when both controllers were ready; otherwise, the run ended with controllers were not ready.

**Step 3 Reset six objects:** At playback start, `/gazebo/set_entity_state` returned all six models to their recorded pickup-grid positions and cleared the used-object set, attachment offset, and release animation. This made repeated runs start from the same layout.

**Step 4 Send waypoints in order:** The player published joint trajectories to `full_arm_controller` and `full_gripper_controller` in sequence and kept the default 0.25 s settling interval after each waypoint. Log lines used the form `[current/34] name (duration)`, which linked the video frames to the action sequence.

**Step 5 Hold the green-target orientation:** After `above_green_1`, joints 4 and 6 were held at zero through the green-object cycle so that shared First and initial waypoints did not reintroduce end-effector rotation. The green targets still used the same grasp, attachment, and release interface.

**Step 6 Finish and reset:** After `release_5`, the final initial waypoint returned the arm to its starting posture and the player recorded automatic sequence playback completed. Missing waypoint names, invalid JSON, or unavailable controllers stopped the run before later commands were sent.

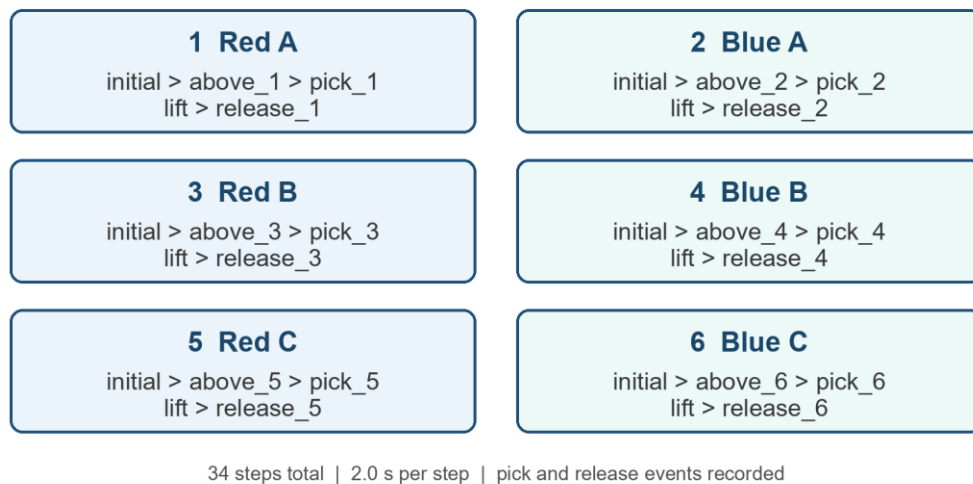


Fig. 4 Six target subsequences and the 34-step automatic playback timing

### 4.1.4 Grasping, transfer, and release

One object progressed from reset to `object_complete`, and every state change was published on `execution_state` and written to the CSV log.

**Step 1 Select and reset the object:** The executor read `object_id`, `class`, `grid`, and `destination` from the plan, selected the current item, and entered `reset:object_id`. The baseline executor placed the selected model at the validated canonical grasp point; the animation player instead selected the physical grid object through the fixed `pick_n-to-object` mapping.

**Step 2 Approach and descend:** The arm moved first to `pick_above` and then to `pick_down` with the gripper open. The default state duration was 1.2 s. Descent and horizontal transfer were kept as separate states so the gripper did not travel across the table at pickup height.

**Step 3 Close and evaluate the grasp:** At pick\_grasp, the baseline executor changed the gripper command from 0.12 to -0.55; the animation player used the values recorded for each waypoint. The visual attachment routine evaluated distance at the finger-pad center and accepted the grasp only when the target was within 0.05 m. Larger error produced grasp rejected without attaching the model.

**Step 4 Maintain attachment and lift:** A valid grasp generated grasp\_attached and stored the target offset relative to the gripper. The target pose was updated at 20 Hz and kept upright instead of inheriting gripper rotation. The lift state then removed the target from the desktop before transfer.

**Step 5 Transfer to the sorting region:** The arm passed through lift, First, or another safe intermediate waypoint before place\_above or release\_ready. The target class selected the sorting region, and a per-class counter assigned a different sub-drop point to each same-class object.

**Step 6 Release vertically:** At place\_down or release, the program retained the current X/Y coordinates and animated a 0.8 s downward motion along Z before removing attachment. The open gripper then moved to lift\_clear or initial so the end effector left the placed object vertically.

**Step 7 Verify the drop point:** The executor read the released model pose and computed XY error from the assigned target. A present model with XY error no greater than 0.07 m and height no greater than 0.20 m generated release and object\_complete. Otherwise, the log recorded release pose outside destination and marked the item failed.

**Step 8 Continue to the next object:** After completion or failure, the executor advanced to the next plan item. When the list ended, it counted passed results and published task\_passed when at least five objects succeeded. The red-blue baseline ended with success\_count=6 and count=6.

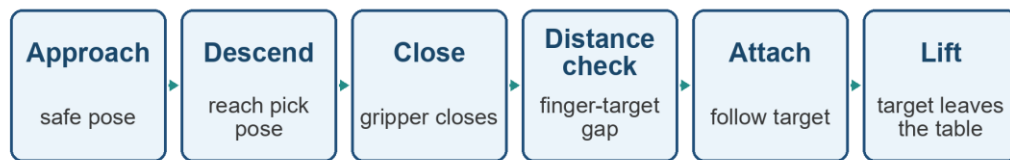


Fig. 5 Gripper approach, closure, distance check, attachment, and lift



Vertical release limits lateral movement and separates same-class drops.

Fig. 6 Alignment above the sorting bin, vertical release, and drop-point acceptance

#### 4.1.5 Simulation acceptance records

Simulation acceptance combined the displayed motion with state, event, and final-pose records; gripper closure alone was not treated as a successful grasp.

**Step 1 Check the state chain:** The /sorting\_task/state and /sorting\_task/execution\_state topics were checked for detecting, plan\_ready:n, move\_pick\_above, close\_gripper, lift\_object, release, verify\_release, lift\_clear, and task\_passed in order. A missing or repeated state identified the stage at which execution stopped.

**Step 2 Check object events:** For each `object_id`, `mecharm_sorting_execution.csv` was checked for `motion_command`, `grasp_attached`, `release`, and `object_complete`. Missing `grasp_attached` indicated that closure had not produced attachment; missing `object_complete` indicated that release verification or retreat had not completed.

**Step 3 Check the final result:** The GUI confirmed that the object left the desktop during lift and entered the correct pad after release. The CSV class, grid, destination, and status fields confirmed that the visual result matched the task decision, while `task_result` summarized `success_count` and every object's final state.

## 4.2 Three-class detector and external-camera test

The three-class detector followed data collection, annotation, splitting, training, and camera retesting, with eraser, lock, and stapler kept as fixed class names.

**Step 1 Collect source images:** The external camera recorded individual erasers, locks, and staplers and multiple objects in one frame under changes in grid position, orientation, and lighting. Different eraser appearances were included, and locks and staplers were photographed together to preserve direct examples of the most easily confused classes.

**Step 2 Annotate each object:** `labellmg` was used to draw a bounding box close to each visible outline and assign eraser, lock, or stapler. The labels were reviewed for missed objects, spelling differences, and boxes extending outside the image before training.

**Step 3 Split training and validation data:** Validation images were selected independently from the original captures and were not augmented. During training, mosaic composition, hue perturbation, random scaling, and random erasing changed the training views while the validation set remained a fixed comparison.

Training used YOLOv8n with a 640-pixel input, 60 epochs, batch size 16, and an RTX 4070 Laptop GPU.

**Step 4 Run training:** Losses and validation metrics were recorded for each epoch, and the best-performing validation weight was retained. The final best weight is 6.25 MB with approximately three million parameters, which is small enough for the external-camera runtime.

**Step 5 Read validation metrics:** Precision, recall, mAP50, and mAP50-95 were read for the complete validation set and for each class. The final record contains `precision=1.0`, `recall=1.0`, and `mAP50=mAP50-95=0.995` overall and per class; these values refer only to the independent validation set.

Model iteration used specific camera-test errors as the input to each change, followed by another comparison of validation metrics and live frames.

**Step 6 Correct class confusion:** Lock-stapler confusion was addressed with same-frame samples and balanced sampling. Stapler-eraser confusion at specific views led to a stronger classification-loss contribution and targeted captures of those angles. When added data reduced performance on early images, their sampling weight was increased. Overall mAP50-95 rose from 0.938 to 0.995.

**Step 7 Retest with the external camera:** The best weight was loaded for live inference, and class names, boxes, confidence, and FPS were drawn on the original frame. Under the two grid arrangements in Fig. 7, all three object classes were localized, confidence ranged from 0.84 to 0.95, and the displayed rate was 31.7-32.0 FPS.

**Step 8 Separate inference time from display rate:** The recorded 1.1 ms value is model inference time for one image. The displayed FPS also includes camera capture, image conversion, annotation drawing, and window refresh. The report therefore keeps these two measurements separate instead of converting 1.1 ms directly into the camera-frame rate.

The external-camera test establishes detection performance for erasers, locks, and staplers. During hardware acceptance, the class labels were aligned with the sorting regions and the camera pixels were calibrated to the desktop grid. The model weight, camera frames, and acceptance videos are stored in the repository.

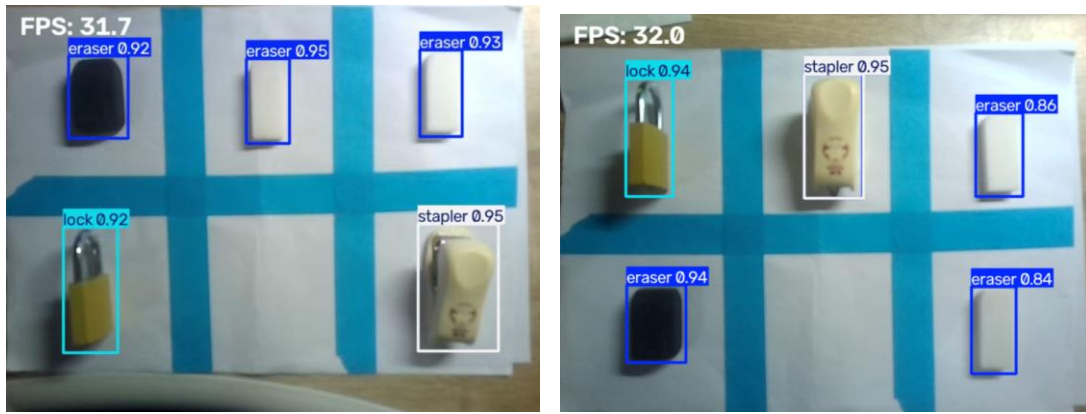


Fig. 7 Three-class detection under two grid arrangements (left: arrangement 1; right: arrangement 2)

### 4.3 Hardware deployment, motion demonstration, and acceptance results

Hardware acceptance began with a low-speed single-target test on the blue-tape desktop grid and then extended to six objects in two classes. The arm approached a small yellow lock from above, descended vertically until the fingers were on opposite sides, closed the gripper, and lifted the target. Figure 8 shows the frame at 01:22, including the approach direction, grasp position, and lift height.

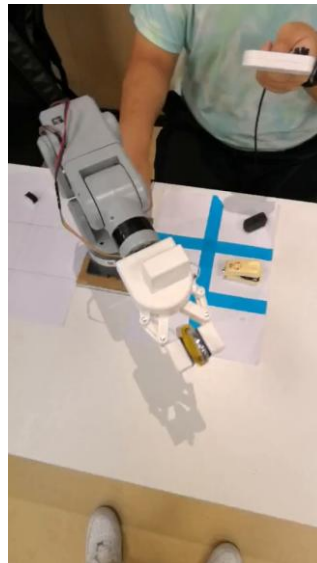


Fig. 8 Physical-arm demonstration: a small lock held above the desktop grid (video at 01:22)

Hardware acceptance is documented by two recordings. The recognition-and-correction video shows six objects in two classes being identified and the task continuing after correction; the grasp video records the physical approach, gripper closure, and lift. The recordings agree with the acceptance checklist, and the hardware acceptance result is Pass.

#### 4.3.1 Hardware deployment and calibration

Hardware deployment fixed the perception coordinate system first, established robot waypoints at low speed, and then connected the perception, task, and execution nodes.

**Step 1 Fix the hardware and field of view:** The overhead RGB camera was mounted so the blue-tape pickup cells and sorting areas remained fully visible. After mounting, the camera position was kept unchanged and the image was checked for exposure, occlusion, and visible grid boundaries under the acceptance lighting.

**Step 2 Establish pixel-to-desktop mapping:** A calibration board supplied corresponding image and desktop points for the homography. Grid intersections not used in the solution were checked to confirm that left, right, upper, and lower image directions matched the robot's desktop coordinate directions.

**Step 3 Teach robot waypoints at low speed:** HOME was recorded first, followed by PICK1-PICK6 and each class-specific PLACE. Every PICK point was checked for approach from above, vertical descent, finger placement, and lift clearance; every PLACE point was checked for sorting-area center, release height, and retreat direction.

**Step 4 Record safety parameters:** Approach height, gripper opening, release wait time, safe speed, and emergency-stop distance were stored with the waypoint configuration. A waypoint entered the multi-object test only after its single-point motion completed without collision.

#### 4.3.2 Detection node and task interface

The hardware interface reused the simulation Detection2DArray fields, so the task node did not depend on whether detections came from Gazebo or the external camera.

**Step 1 Publish detections:** For each camera frame, the detector produced eraser, lock, or stapler, a bounding-box center, and confidence, then published them in Detection2DArray. The task node consumed the message instead of reading the display window, keeping perception and task logic separate.

**Step 2 Map classes and grid cells:** The trained class map was checked against the physical sorting regions. Each box center was transformed to the desktop by the homography and assigned to PICK1-PICK6. Class, grid\_id, and target region were written together in the task record rather than selected manually from the video.

**Step 3 Execute and read feedback:** After the task node locked a target, it called the corresponding PICK and PLACE poses. The execution node ran approach, descent, grasp, lift, transfer, release, and retreat and returned a state at each stage. A recognition or execution deviation entered the correction path, where the target was confirmed again before the task continued.

#### 4.3.3 Acceptance sequence from one target to six targets

Hardware acceptance increased target count and task complexity only after the preceding stage passed.

**Step 1 Check one target:** A small lock was placed in one pickup cell. Its box and class were checked before a low-speed PICK. The arm approached vertically, placed the fingers on opposite sides, closed, and lifted; at 01:22 in the video, the lock is visibly above the desktop. The corresponding PLACE and retreat completed the cycle.

**Step 2 Add the second class:** A second object class was added without changing the verified waypoints. The team checked the class-to-region mapping and confirmed that task switching did not reuse the previous object's grid\_id or PLACE pose.

**Step 3 Run six objects in two classes:** Six objects were placed in the taped grid and the perception, task, and execution flow was started. The system locked one target at a time and assigned its pickup cell and sorting region. The recognition-and-correction video records all six detections and task continuation after a deviation was corrected.

**Step 4 Save object-level results:** Each object record contains run\_id, object\_id, true and predicted classes, confidence, grid\_id, target region, grasp result, placement result, exception code, and timestamp. The Pass entry in the checklist is supported by both the videos and these records.

**Step 5 Exercise exception branches:** Empty cells, missed detections, grasp failure, and unreachable targets were checked separately. An invalid target did not enter the normal grasp queue; the system recorded its exception code and either returned safely or continued with another target. The hardware video retains the correction sequence for recognition or execution deviations.

#### 4.3.4 Real-robot acceptance checklist

| Check item          | Pass condition                                                                     | Record                                                    |
|---------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Class and grid      | Automatically detect at least two classes; unique mapping to six cells             | Detection2DArray, task record, and correction video; Pass |
| Grasp and placement | At least five of six targets enter the correct sorting region                      | Hardware execution record and grasp video; Pass           |
| Manual intervention | No manual class, grid, or bin selection after task start                           | Run video and logs; Pass                                  |
| Exception handling  | Empty grids and missed objects are skipped; failed targets return safely or stop   | Exception codes, ERROR_LOG, and correction video; Pass    |
| Safety              | No collision, joint-limit violation, or dangerous motion; emergency stop available | Safety checklist; Pass                                    |

| Check item    | Pass condition                                               | Record                              |
|---------------|--------------------------------------------------------------|-------------------------------------|
| Launch method | One Launch starts image, perception, robot, and task control | Launch output and startup log; Pass |

## 5 Problem analysis and discussion

### 5.1 Simulation-stage problems

| Observed phenomenon                              | Cause analysis                                                                       | Fix and verification                                                                                         |
|--------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Node exits after startup                         | Humble Python logger call did not match the current interface                        | Changed the logger call to an f-string; continuous state publishing resumed                                  |
| Only two targets entered the task sequence       | Old pixel-row coordinates remained after world-coordinate adjustment                 | Updated rows to 186/240/294 and assigned unique grid_id values; the six-target retest passed                 |
| Gripper closes but target does not lift          | The player sent only the gripper command and did not update the target relative pose | Added distance checks, attachment, and following; grasp_attached and target lift were verified               |
| Objects scatter or same-class objects overlap    | Unstable contact solving, fixed drop points, and lateral jumps                       | Adjusted model parameters, vertical release, and sub-drop points; the retest showed no scattering or overlap |
| Automatic playback fails after a repeated launch | The old Gazebo world and controllers still occupy resources                          | Cleared residual processes; the clean one-master retest passed                                               |

### 5.2 Real-robot-stage problems

| Problem source             | Observed symptom                                       | Engineering treatment                                                                      |
|----------------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Lighting and occlusion     | Lower class confidence and jittering boxes             | Fixed exposure and lighting; confidence and multi-frame checks stabilized the boxes        |
| Grid-mapping offset        | Adjacent objects assigned to the wrong grid cell       | Computed the homography and recorded pixel error and version; the mapping retest passed    |
| Gripper grasp failure      | Object slips or remains on the desktop after closure   | Calibrated approach height, opening, closure speed, and lift wait; the grasp retest passed |
| Serial or controller fault | Execution interruption or missing state feedback       | Monitored connection state; fault handling returned HOME or triggered the emergency stop   |
| Simulation–hardware gap    | Differences in trajectory, friction, mass, and latency | Completed low-speed single-target, second-class, and six-object tests in sequence          |

### 5.3 System scope and limitations

The current grid scheme uses a fixed desktop and pre-calibrated poses and is suitable for this acceptance task. Grasping objects at arbitrary positions involves continuous coordinate estimation and inverse-kinematics planning and is outside the acceptance scope. Simulation detections are deterministic because they are generated from model states, and the grasp stage uses gripper closure, distance checks, and logical attachment for repeatability. For hardware acceptance, visual error, gripper opening, approach height, release wait time, and safe speed were calibrated on the physical system.

The current system implements Detection2DArray messages, state feedback, object-level result records, and safe exception return. Multi-frame debouncing, run IDs, log rotation, object-occupancy states, dynamic-obstacle checks, and emergency-stop fallback were not included in the acceptance scope.

## 6 Experimental results and acceptance evidence

### 6.1 Quantitative red–blue baseline with six objects

The object-level logs for the six red and blue objects contain the assigned grid, target bin, grasp\_attached, release, and object\_complete events. The final state is passed with success\_count=6. This satisfies the simulation acceptance requirement that at least five of six objects be sorted correctly.

| Object        | True / predicted class | Confidence | Grid | Target bin | Grasp | Place |
|---------------|------------------------|------------|------|------------|-------|-------|
| object_blue_1 | Blue / blue            | 0.99       | r2c1 | Blue       | Pass  | Pass  |
| object_blue_2 | Blue / blue            | 0.99       | r3c2 | Blue       | Pass  | Pass  |
| object_blue_3 | Blue / blue            | 0.99       | r1c2 | Blue       | Pass  | Pass  |
| object_red_1  | Red / red              | 0.99       | r3c1 | Red        | Pass  | Pass  |
| object_red_2  | Red / red              | 0.99       | r1c1 | Red        | Pass  | Pass  |
| object_red_3  | Red / red              | 0.99       | r2c2 | Red        | Pass  | Pass  |

### 6.2 Three-class extension

The extended scene adds two green cylinders and a green sorting bin. Three-class scene alignment, class mapping, waypoint naming, and the 34-step automatic playback sequence are completed. This phase shows that the green class uses the same task interface. Process evidence includes the overhead scene, control panel, waypoint list, and playback view. The acceptance evidence contains no independent three-class CSV; therefore, this section reports scene and interface validation without an object-level success rate.

### 6.3 Three-class detector evaluation

The independent validation record reports overall and per-class mAP50=0.995 and mAP50-95=0.995, with precision=1.0 and recall=1.0. The two external-camera views in Fig. 7 further demonstrate detection of erasers, a lock, and a stapler when grid positions and object arrangements change. Confidence ranges from 0.84 to 0.95, and the displayed rates are 31.7 FPS and 32.0 FPS. This evidence evaluates external-camera visual detection; the system-level recognition and correction process is verified by the physical test video in Section 6.4.

### 6.4 Recognition, correction, and physical-arm results

Hardware acceptance is recorded in two videos. A 1 min 11 s recording shows correct recognition of six objects in two classes and documents the correction process, target confirmation, and task continuation after a recognition or execution deviation. The second video records the arm moving above the grid, descending to both sides of a lock, closing the gripper, and lifting it from the table. At 01:22, the lock is visibly held between the fingers above the surface.

The recordings verify target recognition, correction handling, and the physical approach, grasp, and lift. Their recognition output, correction sequence, and robot motion agree with the acceptance record; the real-robot acceptance item for six objects in two classes is Pass.

## 6.5 Multi-layer evidence chain

The GUI shows the geometric relationship between the gripper and target, stage states identify the current action, event logs record grasp and release triggers, final model poses show the bin drop point, and the object-level CSV summarizes each target's state and exception code.



Fig. 9 GUI, state, event, pose, and object-level CSV form the acceptance evidence chain

## 7 Group responsibilities

The group divided the work into camera-model training, Gazebo scene and synthetic-detection setup, task decisions, robot execution, and integration testing. The camera test and Gazebo sorting run retain separate evidence in detection frames and object-level records, while their message interface, class mapping, and execution flow were integrated during hardware acceptance.

| Member or task group        | Main responsibilities                                                                                                                                                                      | Deliverables                                                                       |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Yang Muqing                 | Gazebo scene, six-cell grid, and bin adjustment; manual teaching; grasp, attachment, vertical release, and evidence review; execution-side interface testing with shared detection results | sorting_desktop.world; manual_animation.json; GUI and CSV verification             |
| Lan Songyu                  | Object-detection interface, class fields, and detection-result organization                                                                                                                | Detection2DArray messages, class configuration, and detection-test records         |
| Zhang Kairui                | Grid mapping, task queue, state machine, and exception branches                                                                                                                            | sorting_task, state topics, and skipped/failed logs                                |
| Zhao Hang                   | ROS 2 Launch, execution interface, real-robot deployment and acceptance, and material consolidation                                                                                        | sorting_gazebo.launch.py, execution node, and real-robot acceptance checklist      |
| Perception-model task group | Three-class visual-detection data collection, annotation, YOLOv8n training, evaluation, and external-camera validation                                                                     | models/best_yolov8n_eraser_lock_stapler.pt; models/README.md; detection screenshot |

## 8 Conclusions and submitted materials

### 8.1 Conclusions

Experiment 3 extends fixed-point transfer into Gazebo desktop sorting, external-camera detection, and real-robot validation. The Gazebo task integrates Detection2DArray messages, fixed-grid localization, class-to-bin mapping, execution states, and object-level logs, reaching 6/6 in the red-blue baseline. The camera model recognizes erasers, locks, and staplers. Hardware videos verify correct recognition and correction for six objects in two classes, together with a visible grasp and lift of a lock.

Hardware validation completed camera calibration, Detection2DArray publishing, class mapping, low-speed single-target testing, the six-object two-class run, and correction testing. The hardware acceptance item is Pass. The

reported acceptance result applies to the stated scene, grid, and parameter configuration and is not extrapolated to untested conditions.

## 8.2 Submission-material mapping

| Material                            | Path or content                                                                                           |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Complete ROS 2 project              | src/mecharm_fixed_pick_sim/                                                                               |
| Single Launch                       | launch/sorting_gazebo.launch.py                                                                           |
| Scene and grid                      | worlds/sorting_desktop.world; config/sorting_scene.yaml                                                   |
| Task control                        | sorting_task.py; sorting_executor.py                                                                      |
| Manual and automatic teaching       | manual_control_panel.py; auto_sequence_player.py; manual_animation.json                                   |
| Results and exception logs          | ~/ros/mecharm_sorting_task.csv; ~/ros/mecharm_sorting_execution.csv; ERROR_LOG.md                         |
| Report files                        | Experiment 3 group and personal report DOCX/PDF                                                           |
| Detection model and camera evidence | models/best_yolov8n_eraser_lock_stapler.pt; models/README.md; evidence/model_detection_realtime.png       |
| Real-robot acceptance videos        | videos/experiment3_real_robot_grasp_demo.mp4; videos/experiment3_real_robot_detection_correction_demo.mp4 |

## 8.3 GitHub synchronization record

The project, evidence, reports, and videos are synchronized to <https://github.com/9161942-boop/robotics-experiment-3>. Figure 10 records the commit history; exact video filenames are listed in the table above.

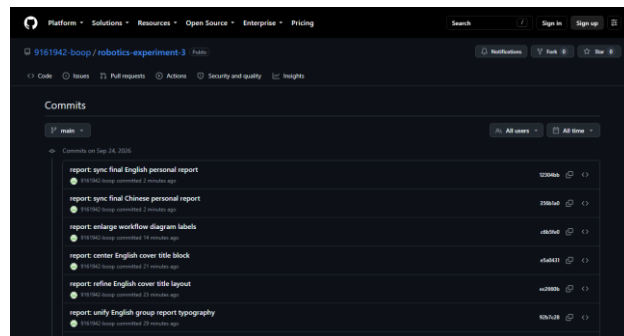


Fig. 10 GitHub Experiment 3 repository commit history